ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 22

Балансировка нагрузки в веб-приложениях

Рассмотрим понятие балансировки нагрузки, настройку Nginx и HAProxy в роли балансировщиков, а также базовые подходы к развертыванию сбалансированных систем с помощью Docker Compose и Kubernetes. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

Балансировка нагрузки (Load Balancing)

Балансировка нагрузки — это механизм распределения входящего сетевого трафика между несколькими серверами или экземплярами приложения. Основная цель балансировки заключается в том, чтобы ни один сервер не был перегружен, а система в целом оставалась производительной, отказоустойчивой и масштабируемой.

В веб-разработке балансировка нагрузки играет важную роль, поскольку позволяет:

- распределять запросы между несколькими экземплярами приложения;
- обеспечивать отказоустойчивость при недоступности одного из серверов;
- увеличивать производительность системы;
- упрощать горизонтальное масштабирование;
- разделять нагрузку между разными типами сервисов, например между статикой и API.

Балансировщики нагрузки можно условно разделить на два основных типа:

Аппаратные балансировщики

- F5;
- Citrix ADC.

Такие решения применяются в крупных инфраструктурах, где требуется высокая производительность, специализированное сетевое оборудование и расширенные функции управления трафиком.

Программные балансировщики

- Nginx;
- HAProxy;
- Traefik;
- Cloudflare.

Программные балансировщики получили широкое распространение благодаря гибкости настройки, доступности и удобству интеграции с современными веб-приложениями.

Сценарии использования балансировки нагрузки

Балансировка нагрузки применяется в различных случаях:

1. **Распределение трафика между серверами** — входящие запросы направляются на разные экземпляры приложения.
2. **Отказоустойчивость и масштабирование** — при сбое одного узла запросы перенаправляются на работающие серверы.
3. **Разделение статики и API** — статика может обслуживаться отдельным сервером, а API — отдельной группой серверов или микросервисов.

Таким образом, балансировка нагрузки является важным инструментом построения современных отказоустойчивых и масштабируемых веб-систем.

Настройка Nginx как балансировщика нагрузки

Базовые алгоритмы балансировки

Nginx поддерживает несколько алгоритмов распределения запросов между серверами.

1. Round Robin
Это алгоритм, используемый по умолчанию. Запросы последовательно распределяются между серверами по кругу.

2. Least Connections
Запрос направляется на сервер с наименьшим количеством активных соединений. Такой подход полезен, если нагрузка на запросы может быть неравномерной.

3. IP Hash
Клиентский IP-адрес используется для определения сервера, на который будет направлен запрос. Это позволяет закрепить конкретного клиента за одним и тем же сервером.

Практика: запуск нескольких backend-серверов

Для демонстрации балансировки можно запустить несколько backend-серверов на разных портах. Например, два основных сервера и один резервный.

Пример простого backend-сервера на Node.js:

```
const express = require("express");

const app = express();
const PORT = process.env.PORT || 3000;

app.get("/", (req, res) => {
  res.json({
    message: "Response from backend server",
    port: PORT
  });
});

app.listen(PORT, () => {
  console.log(`Server started on port ${PORT}`);
});
```

Например, можно запустить серверы на портах `3000`, `3001` и `3002`.

Конфигурация Nginx для балансировки

Для настройки балансировки в Nginx используется блок `upstream`, внутри которого перечисляются backend-серверы.

```
upstream backend {
    server 127.0.0.1:3000;          # Сервер 1
    server 127.0.0.1:3001;          # Сервер 2
    server 127.0.0.1:3002 backup;   # Резервный сервер
}

server {
    listen 80;

    location / {
```

```
        proxy_pass http://backend;
    }
}
```

В данной конфигурации:

- запросы распределяются между серверами `3000` и `3001`;
- сервер `3002` выступает как резервный и используется при недоступности основных серверов;
- все входящие запросы на порт `80` проксируются в группу `backend`.

Тестирование балансировки

После запуска Nginx и backend-серверов можно проверить работу балансировки с помощью `curl` или Postman.

Пример команды:

```
curl http://localhost/
```

При повторных вызовах ответа можно увидеть, что запросы обрабатываются разными backend-серверами.

Health Checks и отказоустойчивость

Одной из важных возможностей балансировщика является исключение недоступных серверов из обработки запросов.

В Nginx для этого можно использовать параметры `max_fails` и `fail_timeout`:

```
upstream backend {
    server 127.0.0.1:3000 max_fails=2 fail_timeout=30s;
    server 127.0.0.1:3001 max_fails=2 fail_timeout=30s;
    server 127.0.0.1:3002 backup;
}
```

В данной конфигурации:

- если сервер дважды подряд не отвечает корректно;
- он считается недоступным;
- на протяжении `30` секунд Nginx не будет направлять на него запросы.

Это позволяет повысить отказоустойчивость приложения и минимизировать влияние сбоев отдельных backend-узлов.

Альтернативные балансировщики: HAProxy

Установка и базовая настройка

Другим популярным программным балансировщиком является **HAProxy**. Он широко используется в высоконагруженных системах благодаря высокой производительности и широким возможностям настройки.

Пример базовой конфигурации HAProxy:

```
frontend http_front
    bind *:80
    default_backend http_back

backend http_back
    balance roundrobin
    server server1 127.0.0.1:3000 check
    server server2 127.0.0.1:3001 check
```

В этом примере:

- `frontend` принимает входящие HTTP-запросы на порту `80`;
- `backend` содержит список серверов приложения;
- используется алгоритм `roundrobin`;
- параметр `check` включает проверку доступности серверов.

Таким образом, HAProxy может выполнять ту же задачу распределения нагрузки, что и Nginx, но при этом часто применяется в системах, где особенно важны высокая производительность и развитые механизмы диагностики.

Оркестрация с Docker и Docker Compose

Развертывание стека с балансировкой

При контейнеризации приложений удобно разворачивать backend-серверы и балансировщик в едином окружении с помощью **Docker Compose** (более подробно будет рассмотрен в следующих практиках).

Пример `docker-compose.yml` для двух backend-сервисов и Nginx:

```yaml
version: "3.8"

services:
  backend1:
    image: node:18
    command: node server.js
    ports:
      - "3000:3000"

  backend2:
    image: node:18
    command: node server.js
    ports:
      - "3001:3000"

  nginx:
    image: nginx
    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
```

Кратко о Kubernetes

Базовые концепции

При дальнейшем развитии инфраструктуры часто используется **Kubernetes** — платформа оркестрации контейнеров.

Некоторые базовые понятия:

- **Pods** — минимальные единицы развертывания, внутри которых работают контейнеры;
- **Services** — объекты, обеспечивающие доступ к группе Pod'ов и распределение трафика между ними;
- **Ingress** — объект для настройки внешнего доступа к сервисам по HTTP/HTTPS.

В Kubernetes балансировка нагрузки может осуществляться на уровне сервиса.

Пример конфигурации `Service`:

```yaml
apiVersion: v1
kind: Service
metadata:
```

```yaml
    name: backend
  spec:
    selector:
      app: backend
    ports:
      - protocol: TCP
        port: 80
        targetPort: 3000
    type: LoadBalancer
```

В данной конфигурации:

- сервис выбирает Pod'ы с меткой `app: backend`;
- внешний трафик приходит на порт `80`;
- внутри контейнера запрос направляется на порт `3000`;
- тип `LoadBalancer` позволяет организовать распределение трафика между несколькими экземплярами приложения.

Таким образом, Kubernetes предоставляет встроенные механизмы масштабирования и балансировки нагрузки, что делает его удобным инструментом для развертывания современных распределённых систем.

Практическое задание

Необходимо реализовать тестовую систему балансировки нагрузки для веб-приложения.

В рамках выполнения задания требуется:

- подготовить не менее двух backend-серверов, работающих на разных портах;
- настроить Nginx как балансировщик нагрузки;
- протестировать распределение запросов между серверами;
- добавить настройки отказоустойчивости через `max_fails` и `fail_timeout`;
- настроить альтернативный пример балансировки через HAProxy.

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой.